

# Introduction to Computational and Systems Genomic

**Week 4#**

- Course Challenge

# The challenge

**A.** Create a classifier between two types of cancer (LUSC and HNSC)

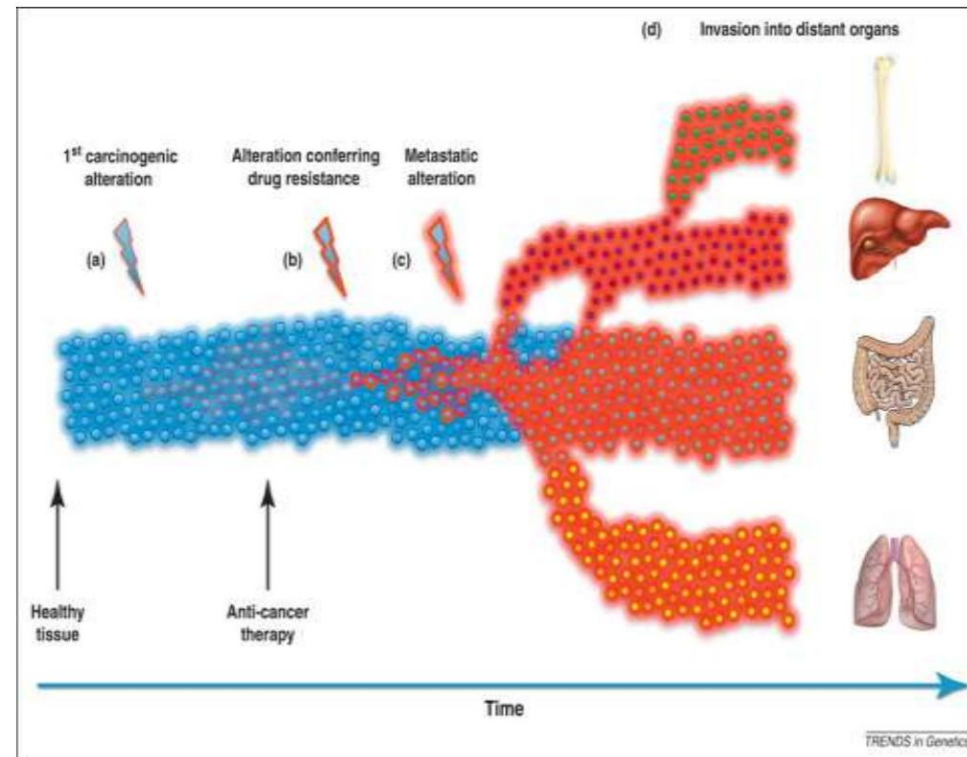
Using genomic information (mutations) from 100 cancer-related genes.

**B.** Create a classifier between two types of cancer that uses genomic information **and methylation levels** from those 100 cancer-related genes.

# Missions!

- **Learn about the concept of selection feature**
  - **Learn about overfitting**
- **Get a feel for working with "big genomic data"**
- An initial sense of working with mutations in different areas of the gene

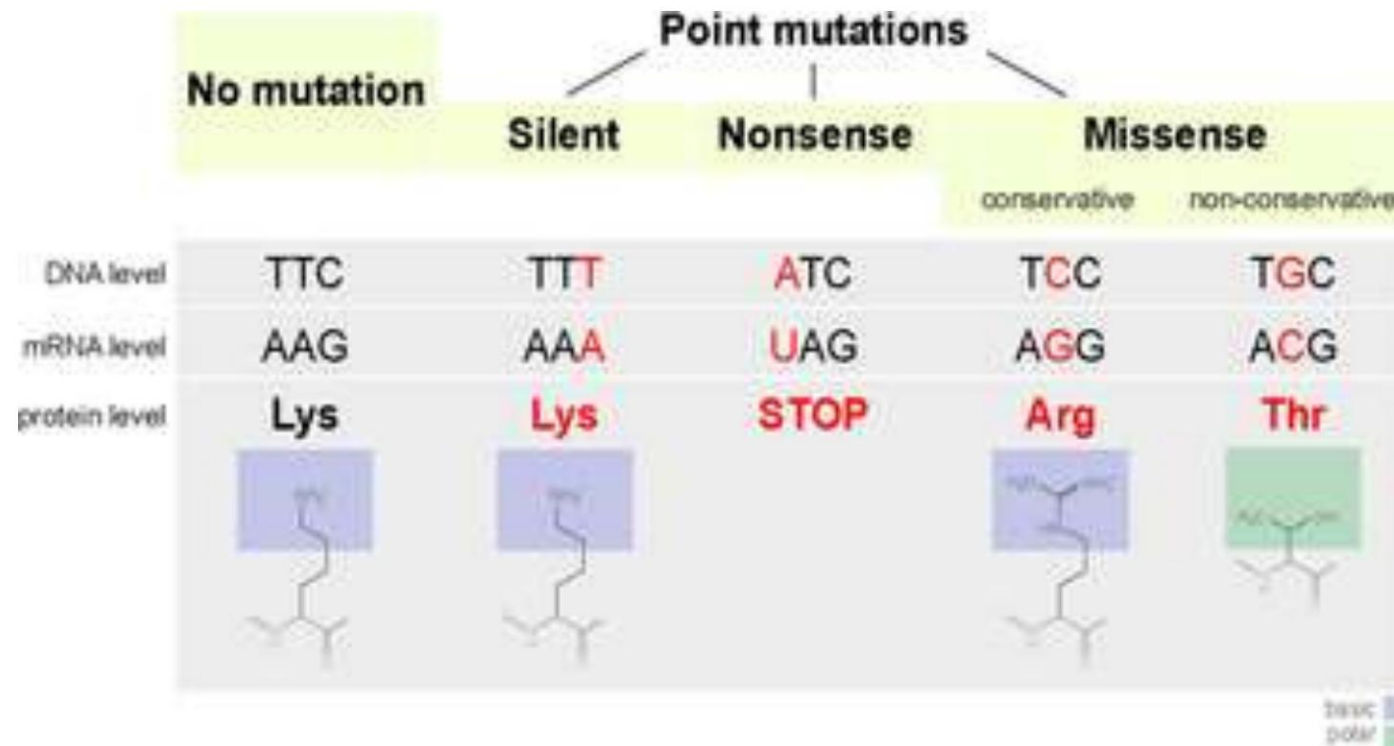
# Introduction to the evolution of cancer



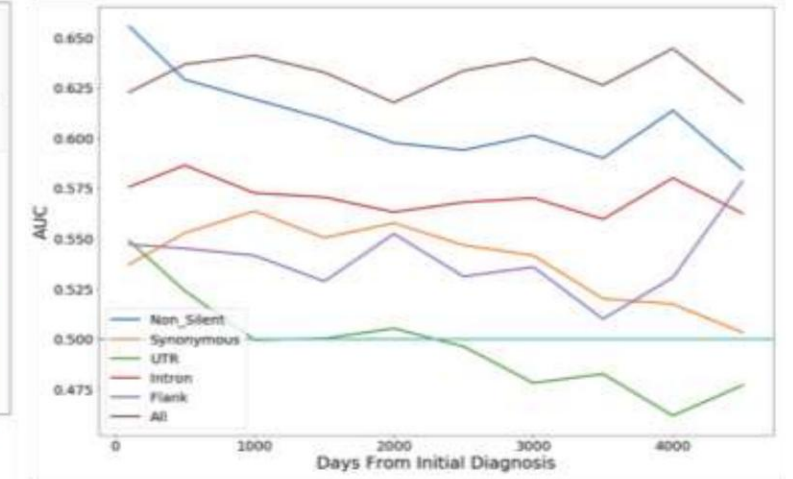
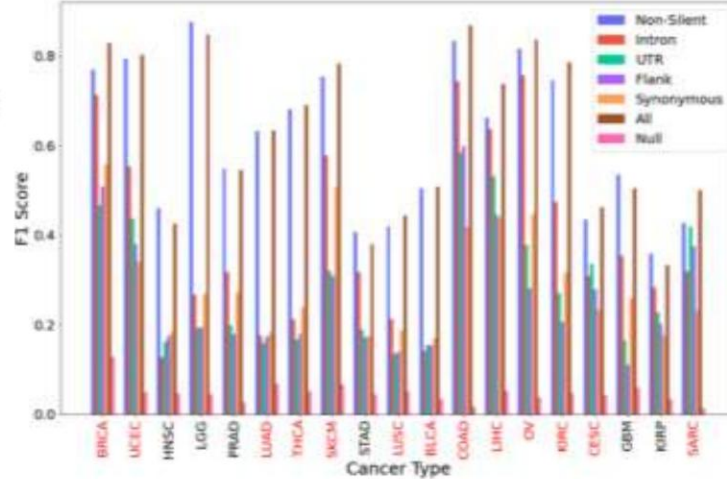
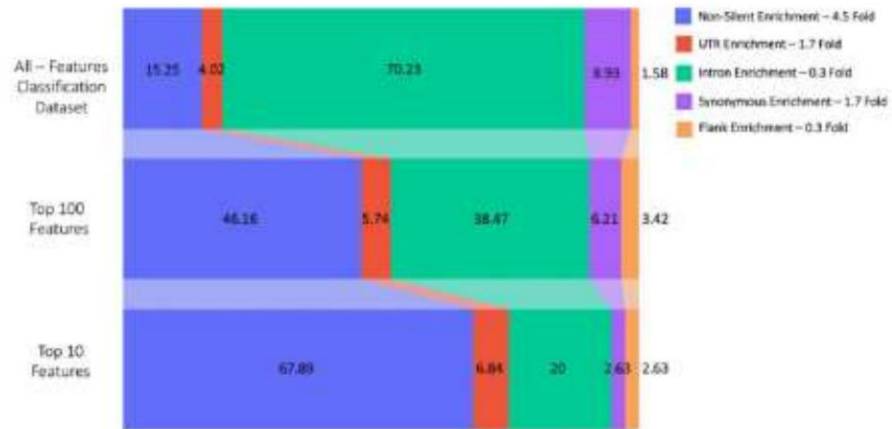
Oncogene – overexpression leads to cancer

Tumor Suppressor Gene – loss of function leads to cancer

# Different types of mutations



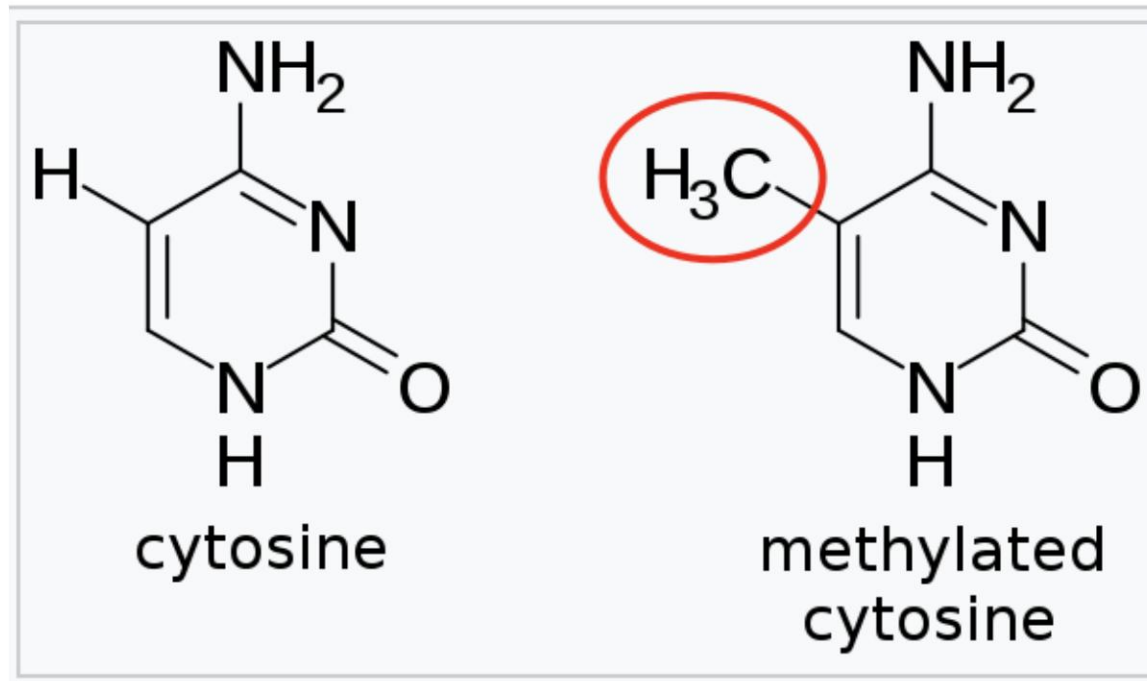
# Silent mutations are predictive!



# Mutations - The Data

| case_id   | Label | Gene_name | Chromosome | Start_Position | End_Position | Mut_Strand | Variant_Classification | Reference_Allele | Tumor_Seq_Allele1 | Tumor_Seq_Allele2 |
|-----------|-------|-----------|------------|----------------|--------------|------------|------------------------|------------------|-------------------|-------------------|
| 13f63218- | 2     | TP53      | chr17      | 7673728        | 7673728      | +          | Nonsense_Mutation      | C                | C                 | A                 |
| 13f63218- | 2     | BRCA2     | chr13      | 32339828       | 32339828     | +          | Missense_Mutation      | G                | G                 | C                 |
| 13f63218- | 2     | APC       | chr5       | 112838253      | 112838253    | +          | Missense_Mutation      | A                | A                 | T                 |
| 13f63218- | 2     | ATM       | chr11      | 108268379      | 108268379    | +          | Intron                 | C                | C                 | G                 |
| 13f63218- | 2     | ATM       | chr11      | 108353791      | 108353791    | +          | Missense_Mutation      | C                | C                 | G                 |

# Methylation



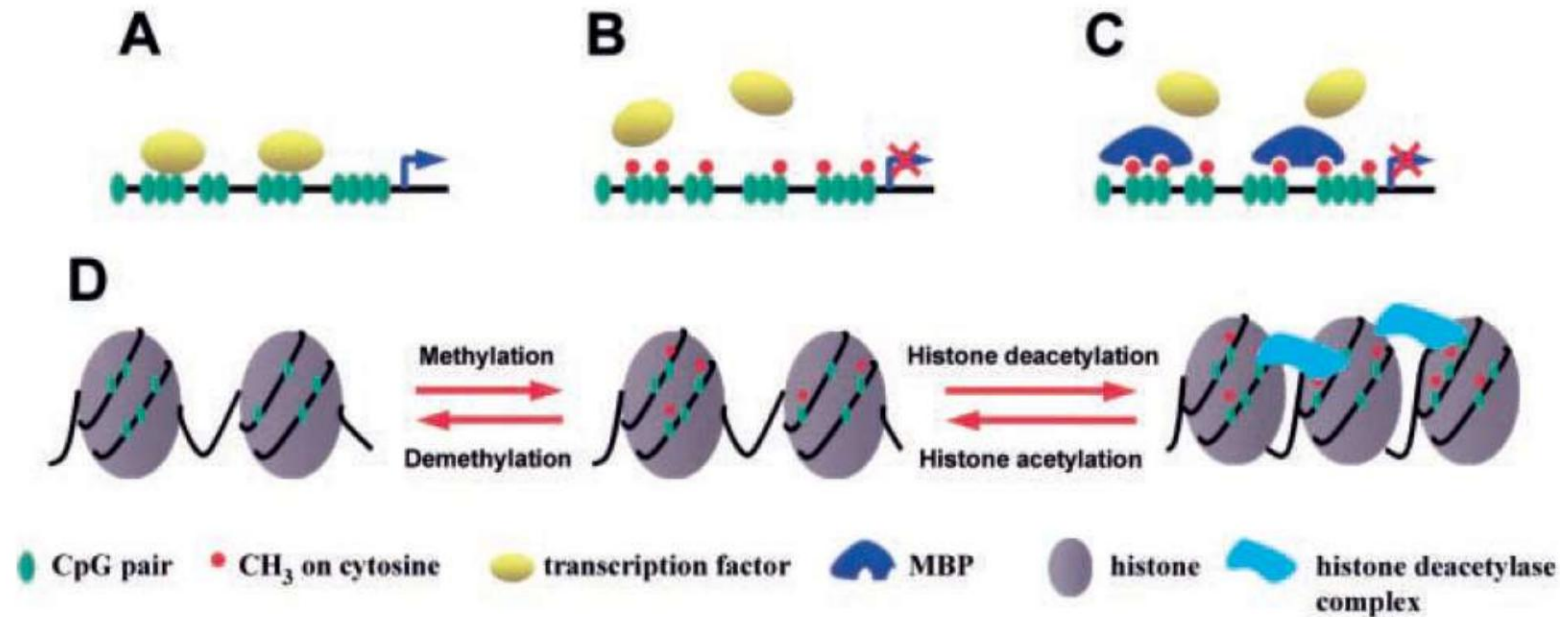


# CpG Islands

For example:

...GCGCGCGTTGCGCGCGCGGC C...

# Methylation represses transcription



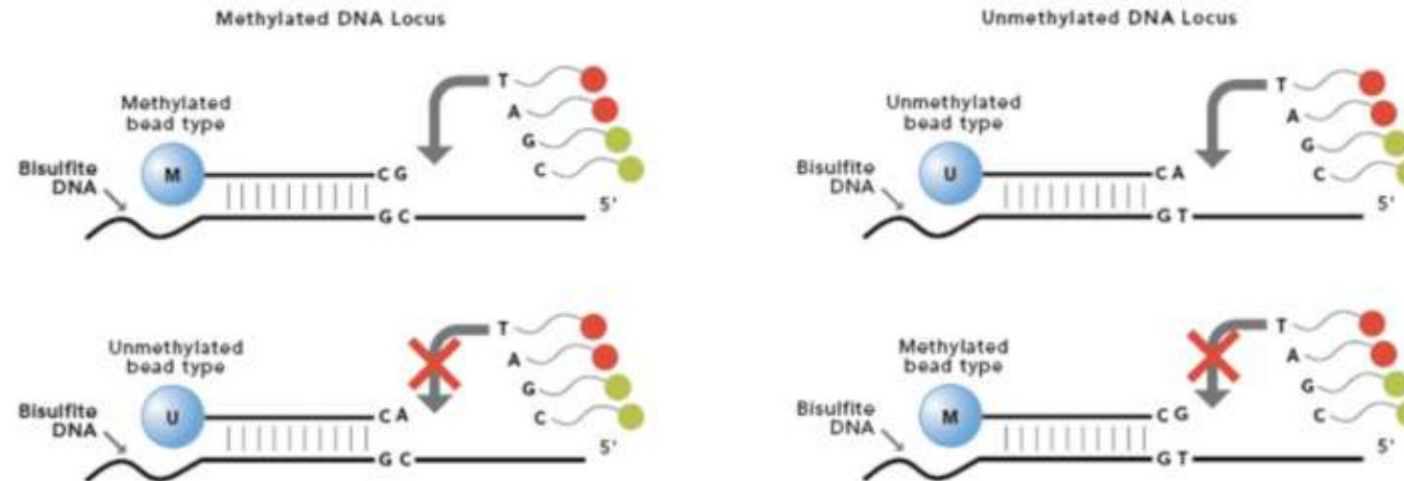
# Methylation - The Data

- Treatment with bisulfite converts unmethylated cytosine (C) to thymine (T), while methylated C\*  
Remains the same
- DNA probes are targeted to specific sites in CpG islands, and we will get more C (than T) if the site was mostly methylated and more T if it was mostly unmethylated.
- M is a measure of the number of times it was methylated, U is a measure of the number of times it was unmethylated:

$$\hat{d}_{i,1} = \frac{M}{M + U}$$

# Methylation - The Data

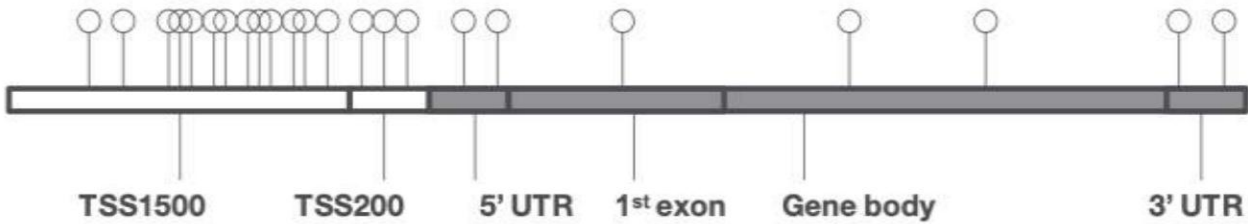
## Infinium Methylation Assay



The Infinium Methylation Assay uses two different bead types to detect CpG methylation. The U bead type matches the unmethylated CpG site; the M bead type matches the methylated site. In the top figure, the unmethylated CpG target site matches with the U probe, enabling single-base extension and detection. It has a single-base mismatch to the M probe, which inhibits extension. If the CpG locus of interest is methylated (bottom figure), the reverse occurs.

# Methylation - The Data

Figure 2: HumanMethylation450 BeadChip Provides Coverage Throughout Gene Regions



| Feature Type | Genes Mapped | Percent Genes Covered | Number of Loci on Array |
|--------------|--------------|-----------------------|-------------------------|
| NM_TSS200    | 14895        | 0.79                  | 2.56                    |
| NM_TS1500    | 17820        | 0.94                  | 3.41                    |
| NM_5'UTR     | 13865        | 0.78                  | 3.34                    |
| NM_1stExon   | 15127        | 0.80                  | 1.62                    |
| NM_3'UTR     | 13042        | 0.72                  | 1.02                    |
| NM_GeneBody  | 17071        | 0.97                  | 8.97                    |
| NR_TSS200    | 1967         | 0.65                  | 1.84                    |
| NR_TSS1500   | 2672         | 0.88                  | 2.92                    |
| NR_GeneBody  | 2345         | 0.77                  | 5.34                    |

# Methylation - The Data

| case_id                              | Label | probeID    | beta_val | CpG_chrm | CpG_beg   | CpG_end   | probe_stra | matching_genes |  |
|--------------------------------------|-------|------------|----------|----------|-----------|-----------|------------|----------------|--|
| 01420c4e-6013-4d3f-87bd-a6cd9e3f8e87 | 2     | cg00008446 | 0.29792  | chr1     | 156845981 | 156845983 | -          | NTRK1          |  |
| 01420c4e-6013-4d3f-87bd-a6cd9e3f8e87 | 2     | cg00016156 | 0.05066  | chr5     | 171388178 | 171388180 | +          | NPM1           |  |
| 01420c4e-6013-4d3f-87bd-a6cd9e3f8e87 | 2     | cg00022858 | 0.08044  | chr4     | 54230448  | 54230450  | -          | PDGFRA         |  |
| 01420c4e-6013-4d3f-87bd-a6cd9e3f8e87 | 2     | cg00031759 | 0.073481 | chr13    | 32315396  | 32315398  | +          | BRCA2          |  |

# Ready-made features

- The number of mutations in each gene (per patient) for the 100 genes  
Selected

You will need to create additional features! For the methylation task, you will need to create all the features.

# Details about the assignment

- The assignment is due by 10.6 (10% of the final grade)
- There is a detailed document in the model on what is required for submission
- Can be done alone or in pairs/threes

A score of 100 for a convincing report... We want to see real experience!

- The groups' predictions will be compared to the real labeling and we will calculate your error on the test

- Bonus for the winners of the competition!
- The winners of the competition will probably explain their methods in class



# Pay attention!

Overfitting • •

The data is noisy/biased

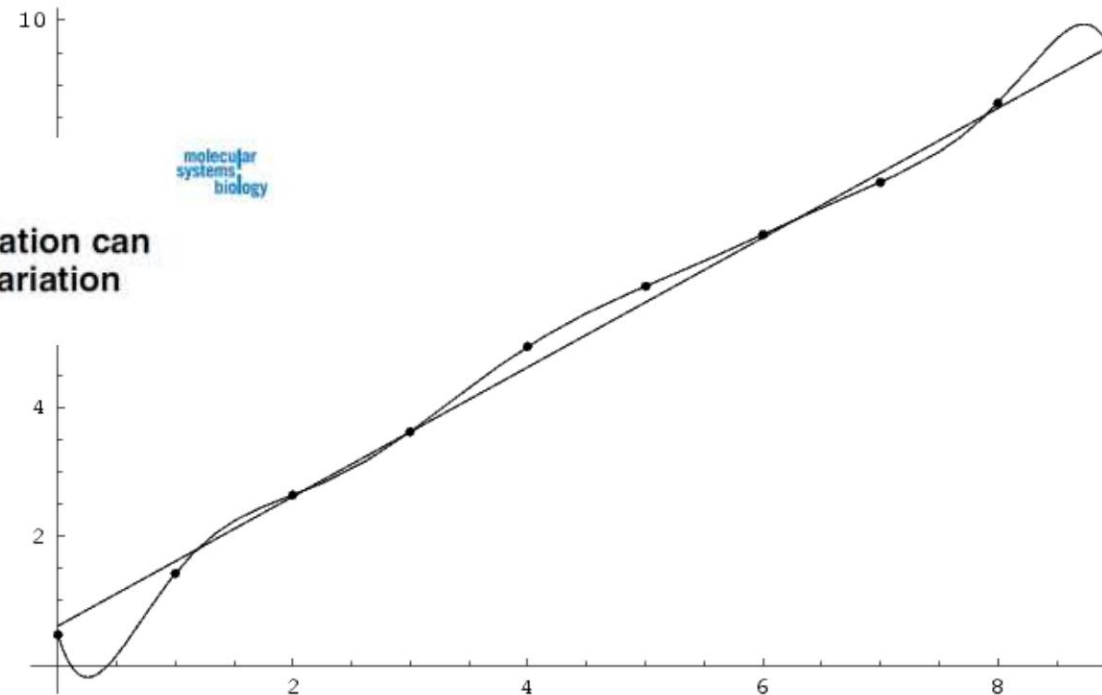
- You can (and should) think of your own features •

You can use complicated statistical models, but in our experience  
it is hard to beat a simple model + knowledge of biology  
+ knowledge of statistics\overfitting

# Overfitting

Molecular Systems Biology 6: Article number 400; doi:10.1038/msb.2010.59  
Citation: *Molecular Systems Biology* 6:400  
© 2010 EMBO and Macmillan Publishers Limited. All rights reserved. 1744-4292/10  
www.molecularsystemsbiology.com

**Sequence signatures and mRNA concentration can explain two-thirds of protein abundance variation in a human cell line**



Noisy (roughly linear) data is fitted to both linear and [polynomial](#) functions. Although the polynomial function passes through each data point, and the linear function through few, the linear version is a better fit. If the regression curves were used to extrapolate the data, the overfit would do worse.

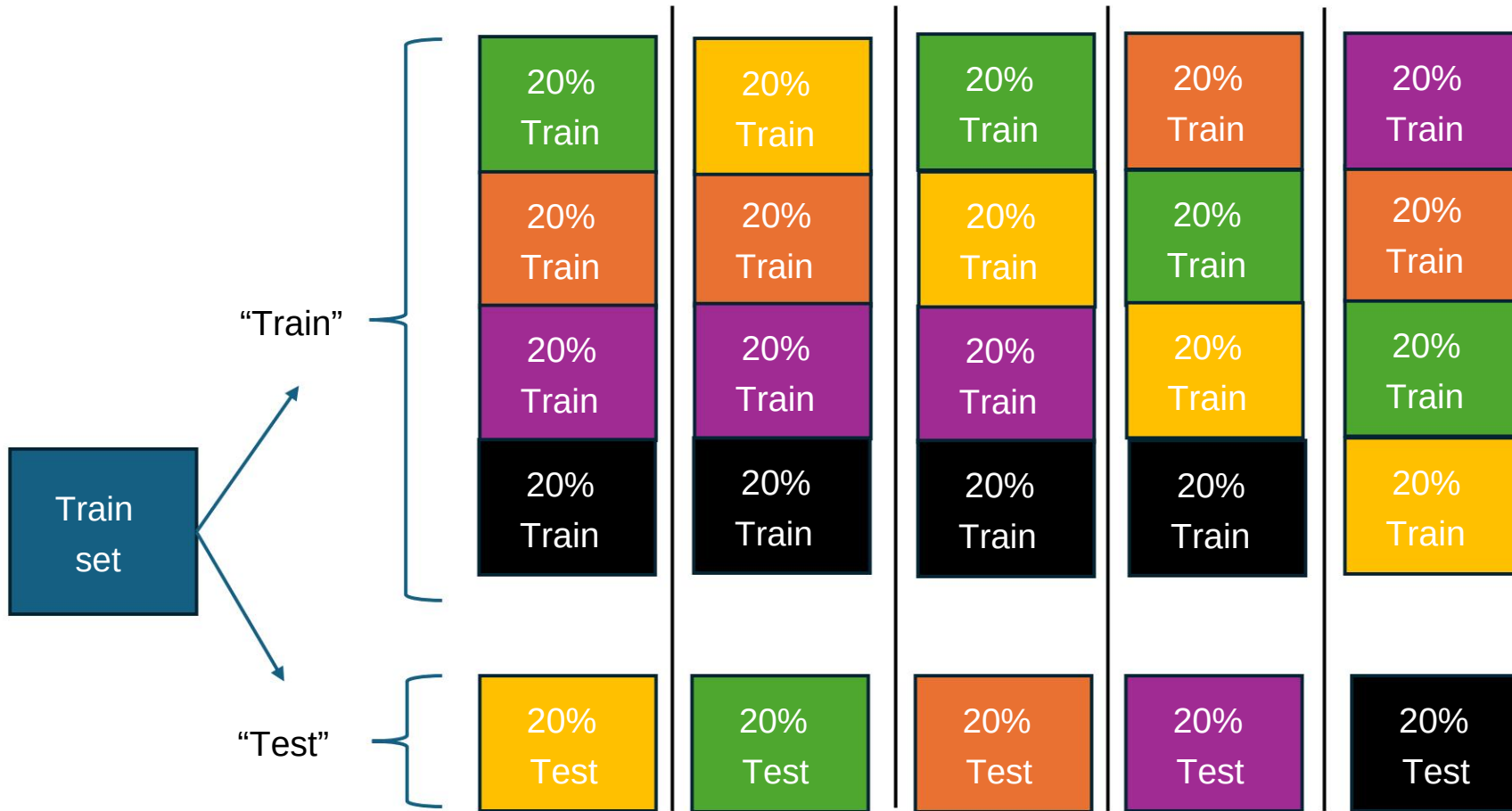
**Any idea how to deal with it ?**

# How to deal with overfitting

- Cross validation – evaluation via train/test..
- Regularization -- penalizing models with too many features
- Pruning – a phase of increasing the modeling after “growing” it
- Minimum description length – optimize the log-likelihood + the model complexity
- Adjusted correlation.

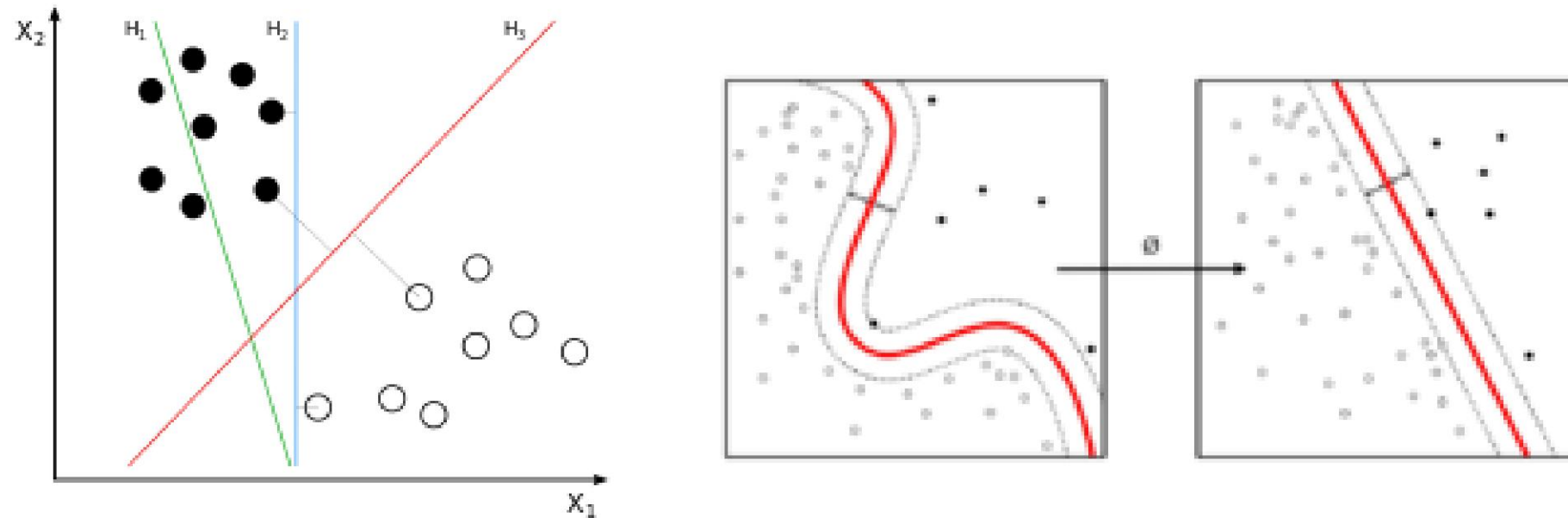
**The approaches are strongly related**

# Cross validation



- The idea is to evaluate the improvement through the test and not through the train.

# Support Vector Machine – SVM



Later...

# Initial proposal for the stages of the task

- 1) Produce a lot of features
- 2) Use the known data (the training set) to select features and find the model weights
- 3) Use the model on the unknown data (the test set) to predict the type of cancer

# Example

- known:

G1 ATG AAA GAG TCT CGC CCC TAG CCA .. C1  
G2 ATT AAA GAG TTT CGC CCC TAG CCA .. C1  
G3 ATG AAA GAG TTT CGC CCC TAG CCA .. C2  
G4 ATG AAA GAG TTT CGC CCC TAG CCA .. C1  
G5 ATT AAA GAG TCT CGC CCC TAG CCA .. C2  
G6 ATG AAA GAG TCT CGC CCC TAG CCA .. C2  
G7 ATG AAA GAG TTT CGC CCC TAG CCA .. C1  
G8 ATG AAA GAG TTT CGC CCC TAG CCA .. C2

Cancer type



- Unknown:

G9 ATG AAA GAG TCT CGC CCC TAG CCA ..  
G10 ATT AAA GAG TTT CGC CCC TAG CCA ..

# 1) Produce features

• known:

G1 ATG AAA GAG TCT CGC CCC TAG CCA ..

G2 ATT AAA GAG TTT CGC CCC TAG CCA ..

G3 ATG AAA GAG TTT CGC CCC TAG CCA ..

G4 ATG AAA GAG TTT CGC CCC TAG CCA ..

G5 ATT AAA GAG TCT CGC CCC TAG CCA ..

G6 ATG AAA GAG TCT CGC CCC TAG CCA ..

G7 ATG AAA GAG TTT CGC CCC TAG CCA ..

G8 ATG AAA GAG TTT CGC CCC TAG CCA ..

• Unknown:

G9 ATG AAA GAG TCT CGC CCC TAG CCA ..

G10 ATT AAA GAG TTT CGC CCC TAG CCA ..

Mutation G->T in position 3 Mutation T->C in position 11

|   |   |     |
|---|---|-----|
| 0 | 1 | ... |
| 1 | 0 |     |
| 0 | 0 |     |
| 0 | 0 |     |
| 1 | 1 |     |
| 0 | 1 |     |
| 0 | 0 |     |
| 0 | 0 |     |

Many additional features



# Feature Selection )2

- Divide the known set to 3 subsets:

Known Train (40%):

G1 ATG AAA GAG TCT CGC CCC TAG CCA .. C1

G2 ATT AAA GAG TTT CGC CCC TAG CCA .. C1

G3 ATG AAA GAG TTT CGC CCC TAG CCA .. C2

Cancer type



Known Validation (40%):

G4 ATG AAA GAG TTT CGC CCC TAG CCA .. C1

G5 ATT AAA GAG TCT CGC CCC TAG CCA .. C2

G6 ATG AAA GAG TCT CGC CCC TAG CCA .. C2

Known Test (~20%):

G7 ATG AAA GAG TTT CGC CCC TAG CCA .. C1

G8 ATG AAA GAG TTT CGC CCC TAG CCA .. C2

# Find the **best first** feature based on the **"Validation" set (which you chose from the training set)**

Test all features separately: • Train

an SVM with a single feature (we will explain in more detail what this is later) •

Test the same SVM on the validation set (from the known set) by calculating

The error

Choose the feature with the best **performance** on the "validation" set. For example:

\*

Cancer type  
↓  
Predicting cancer type ↙

G4 ATG AAA GAG TTT CGC CCC TAG CCA .. C1 C1 G5  
 ATT AAA GAG TCT CGC CCC TAG CCA .. C2 **C1** G6 ATG  
 AAA GAG TCT CGC CCC TAG CCA .. C2 C2

Error = 33% = 1E

Find the **second best feature** based on the "Validation" set (**which you chose from the training set**)

Let's say the first best feature was 3 position in T>-G Mutation (which we will call "Feature 1") which gives the lowest error (1E)

Now we will look for the next feature. For each feature that is not feature 1:

- Run an SVM on the training set (as before) whose features are feature 1 and the feature under test
- Calculate the error of that SVM on the "validation" set (from the Known set)

Find the **second best feature** based on the "Validation" set **(which you chose from the training set)**

Let's say the second best feature was "11 position in C>-T Mutation" (Feature 2) which gives the smallest error on the "validation" set (from the Known set).  
And its value is  $2E$

Only if  $1E < 2E$  will we continue the search for a third feature. Otherwise, we will move on to the final step in Feature Selection.

## Find the **third** best feature based on the "Validation" set (which you chose from the training set)

**Let's assume** that  $2E$  is indeed less than  $1-E$ . So we'll look for the third feature similarly to last time:

For any feature that is not a 1\2 feature

- SVM on the training set (as before) whose features are feature 1, feature 2 and the test feature
  - Calculate the error of that SVM on the "validation" set (from the known set)

## Find the **third** best feature based on the "Validation" set (which you chose from the training set)

Let's say the third best feature is 1000 position in G>-T Mutation, and that-  
The SVM that uses it (and features 1 and 2) gives the error 3E.

Only if  $2E < 3E$  will we continue the search for a fourth feature. If  $2E \geq 3E$  then we will stop the search and move on to the final stage in Feature Selection.

# Selection Feature – Last step

Suppose we stopped after trying to find a third feature and it turned out that for the third best feature,  $2E \geq 3E$  holds, meaning that the SVM with features 1,2,3 gives a larger error than the SVM with features 1,2.

So we'll take the SVM with those 2 features and calculate its error on the "test" set from the known set.

Known Test

G9 C1'

G10 C2'

# Selection Feature – Last step

If the error on the known "test" set turned out to be small (or similar) to the error we got on the "validation" set from the known set and is good enough for us –

We will take this model and use it to predict the set Unknown.

If the error is not good on the "test" set than on the known set, try using a different method for selecting features that might lead to fewer features and use the methods mentioned to deal with overfitting (read about them).

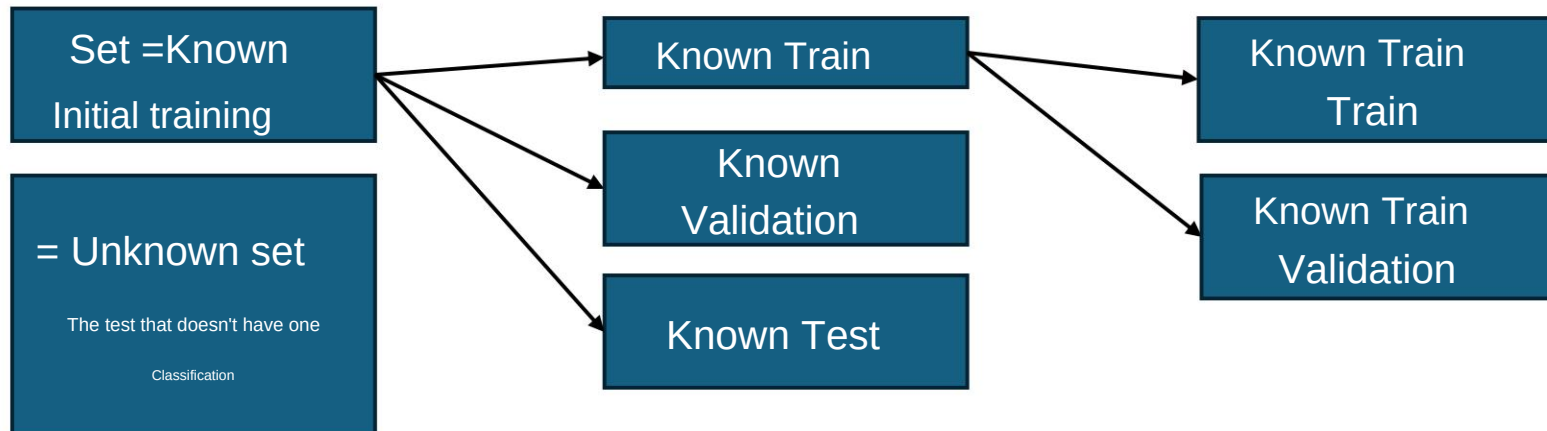
\*It could simply be that the classification is too difficult with the current data... (In the current challenge it is not like that ÿ)



# Selection Feature – Another Nuance

If our model has hyper-parameters that we want to choose, it is worth splitting the known train validation and known train training into known train and also known train.

You need to train the known train and check the resulting error with different hyperparameter options on the known validation train (and then choose the best hyperparameters). Sometimes this is not possible because there is not enough data.



# Cross Validation + Feature Selection

A good way to avoid overfitting is to do cross validation (split into 5 known and unknown, for example) and then in each split do Feature Selection on the known.

The final model can be: A.

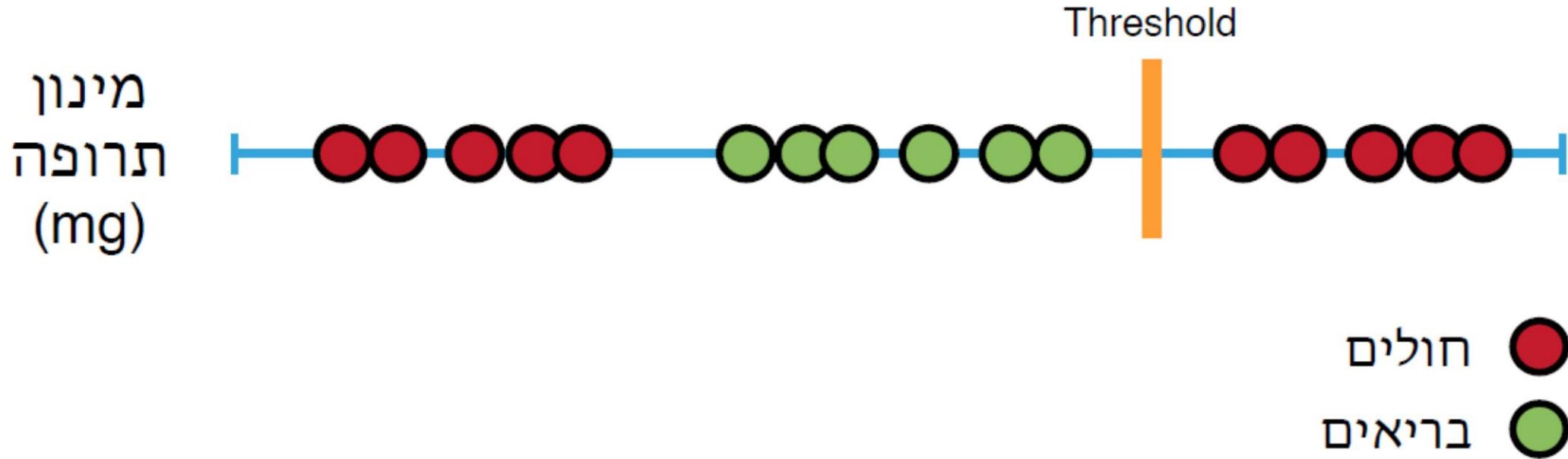
A joint vote of all models from the cross-validation B. The  
best model among all models from the cross-validation C.

A model trained on all data with the most common features+hyperparameters in each  
model from the cross-validation (this could be problematic... why?)

# Support Vector Machine – SVM

- Supervised Machine Learning Algorithm
  - Used for classification (or sometimes called sorting)
- It can be used to classify data that cannot be separated linearly.

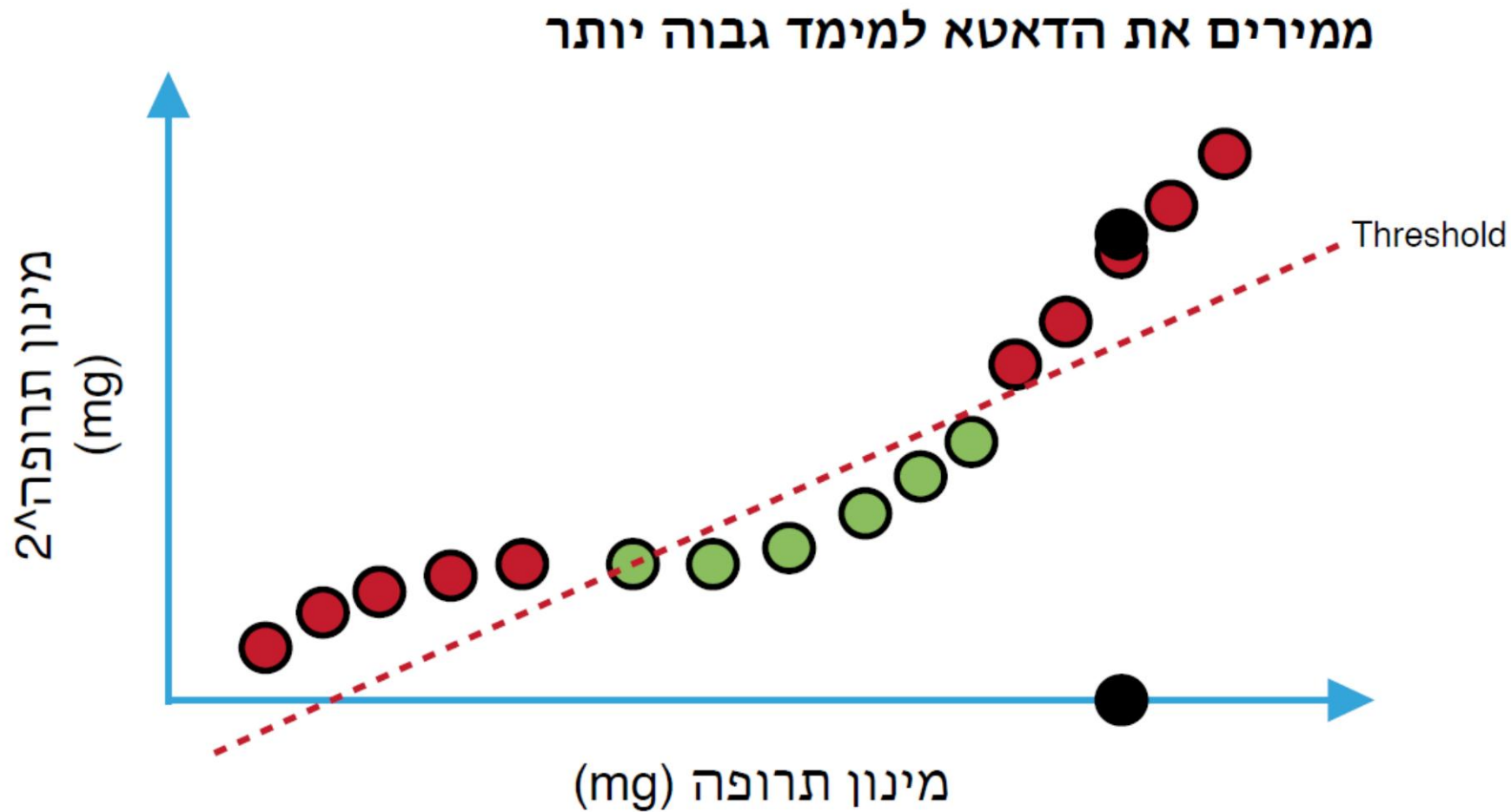
# Support Vector Machine – SVM



Can a better threshold be found?

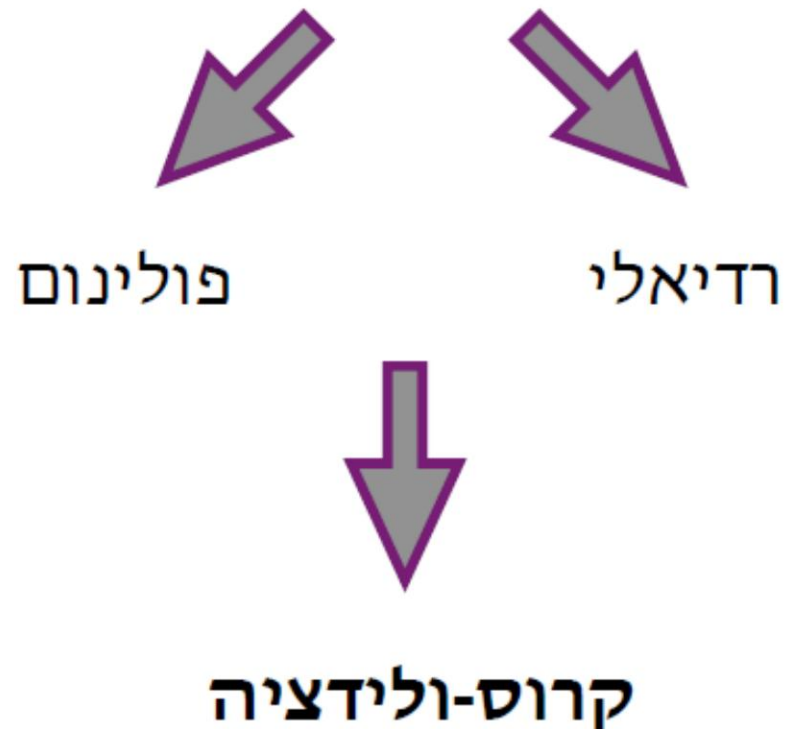
yes!!

# Support Vector Machine – SVM



# Support Vector Machine – SVM

**Kernel functions**



(To choose the best hyperparameter)

## The Challenge – A Proposal for a Summary Process

1. Feature Generation - Create new features (think of ideas, read articles, online)
2. Cross Validation - Split the training set given to you into K splits (try several options and continue with the next step each time) of "Known"  
"Unknown" "Always the same thing"
3. Selection Feature – Make any Selection Feature on a set  
The Known (for example, as described in the lecture)
4. Use a final model that uses the features (and hyperparameters) you found to classify the TEST set (for which you do not have the classification). The model could be, for example, the joint vote of the models built in cross-validation.

# Feature Generation

| Case id | Gene name | chromosome | Start position | End position | Reference Allele | Tumor Seq Allele | Variant classification |
|---------|-----------|------------|----------------|--------------|------------------|------------------|------------------------|
| Yossi   | IDH1      | 1          | 300            | 300          | G                | A                | 5'UTR                  |
| Yossi   | BRCA1     | 3          | 350            | 351          | AT               | --               | Frame shift deletion   |
| Hanna   | IDH1      | 1          | 303            | 303          | T                | C                | 5'UTR                  |
| Leah    | TP53      | 7          | 715            | 715          | C                | G                | Intron                 |
| Leah    | IDH1      | 1          | 300            | 300          | G                | A                | 5'UTR                  |



# Feature Generation

Possible feature: Do patients have the specific mutation or not (we also need to define what a specific mutation is in our eyes? Same location? Exactly the same change at the same location? Here in the example - exactly the same nucleotide change at the same location)

| Chr7 – 715:715<br>– C to G | Chr1 – 303:303<br>– T to C | Chr3 – 350:351 –<br>-- AT to | Chr1 – 300:300 –<br>G to A | Patient/ Feature |
|----------------------------|----------------------------|------------------------------|----------------------------|------------------|
| 0                          | 0                          | 1                            | 1                          | Yossi            |
| 0                          | 1                          | 0                            | 0                          | Hanna            |
| 1                          | 0                          | 0                            | 1                          | Leah             |

# Data division (single split)

```
df_known_temp, df_known_test = train_test_split(
    df_known,
    test_size=0.2,
    stratify=df_known['Label'],
    random_state=0)
df_known_train, df_known_validation = train_test_split(
    df_known_temp,
    test_size=0.5,
    stratify=df_known_temp['Label'],
    random_state=0)
print(f'Known Train {len(df_known_train)}:', Counter(df_known_train.Label))
print(f'Known Validation {len(df_known_validation)}:', Counter(df_known_validation.Label))
print(f'Known Test {len(df_known_test)}:', Counter(df_known_test.Label))
```

```
Known Train 257: Counter({2.0: 130, 1.0: 127})
Known Validation 258: Counter({2.0: 131, 1.0: 127})
Known Test 129: Counter({2.0: 65, 1.0: 64})
```

# Cross Validation

```
from sklearn.model_selection import train_test_split
def split_data(df, rand_i):
    # Function that splits data into known train, validation, test
    ### Input:
    # df - Dataframe of known data
    # rand_i - random state setter
    ### Outout:
    # df_known_train, df_known_validation, df_known_test - Dataframes of known train, validation, test
    df_known_temp, df_known_test = train_test_split(df_known,
        test_size=0.2,
        stratify=df_known['Label'],
        random_state=rand_i)
    df_known_train, df_known_validation = train_test_split(df_known_temp,
        test_size=0.5,
        stratify=df_known_temp['Label'],
        random_state=rand_i)
    return df_known_train, df_known_validation, df_known_test
```

# Cross Validation

```
k_splits = 5
for i in range(k_splits):
    df_known_train, df_known_validation, df_known_test = split_data(df, i)
    #Feature Selection
```

# Feature Selection

```
from sklearn.svm import SVC
feats_chosen = ['Mutations_in_ABL1', 'Mutations_in_AKT1']
X_train = df_known_train[feats_chosen]
y_train = df_known_train['Label']
X_val = df_known_validation[feats_chosen]
y_val = df_known_validation['Label']
# Initialize and train the SVM model
svm = SVC(kernel='poly', C=1, degree= 2, random_state=42)
svm.fit(X_train, y_train)
# Predict on the validation set
y_pred = svm.predict(X_val)
error = sum(y_pred!=y_val)/len(y_val)
```

Run each time with a different feature and add features that remove the error.

# Selection Feature - Hyperparameter Selection

| Parameter           | Description                                                                                                                                        | Applies to Kernel(s) |
|---------------------|----------------------------------------------------------------------------------------------------------------------------------------------------|----------------------|
| <code>C</code>      | Regularization parameter (controls margin vs. misclassification trade-off)                                                                         | All                  |
| <code>kernel</code> | Type of kernel function: <code>'linear'</code> , <code>'poly'</code> , <code>'rbf'</code> , <code>'sigmoid'</code> ,<br><code>'precomputed'</code> | All                  |
| <code>degree</code> | Degree of the polynomial kernel function                                                                                                           | <code>'poly'</code>  |

\*Partial list for SVM

You can use, for example, GridSearchCV  
or RandomizedSearchCV or write code that checks this directly.

# Cross Validation + Feature Selection

- You can use code from the internet that does some or all of the things together, and may use other methods for selecting features
- **It is very important that you understand what the code does and why!**



# Final model

```
X_train = df_train[best_feats]
y_train = df_train['Label']
best_hps = {'kernel':'poly','degree':2, 'C':0.001}
final_svm = SVC(kernel=best_hps['kernel'], C=best_hps['C'], degree= best_hps['degree'], random_state=42)
final_svm.fit(X_train, y_train)
y_pred = final_svm.predict(df_test[best_feats])
df_test.loc[:, 'predict_label'] = y_pred
df_submit = df_test[['case_id', 'predict_label']]
df_submit.head()
```

|   | case_id                              | predict_label |
|---|--------------------------------------|---------------|
| 0 | 03b57fee-55c7-4873-b7ef-e29abd98863a | 2.0           |
| 1 | 0425cb44-66af-4e25-afcb-cacccc2f1179 | 2.0           |
| 2 | 044579ef-16a6-4f00-b951-edc423d8a14f | 2.0           |
| 3 | 05d4d9c7-fb6f-439a-bb97-f0e2737a7773 | 2.0           |
| 4 | 0858c8b7-e2eb-4461-b65e-9d476029ad8d | 2.0           |

Note that it was possible to choose to train several models (cross-validation) and choose the option that received the most votes.

From the same models